

Reviewer Pack — Hypercontractive Laplacian-Spectrum Flatness Bounds Spectral...

Entry c8a7a640d375 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 16:22:18 UTC

Hypercontractive Laplacian-Spectrum Flatness Bounds Spectral SOS Max-Cut Ratio

Entry ID: c8a7a640d375 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 16:22:18 UTC

1. Conjecture

Field A (mathematical branch): Fourier-analytic hypercontractive flatness invariants of graph Laplacian spectra (4-norm-to-2-norm ratios in the Bonami-Beckner-Bourgain tradition, applied as a ‘pseudo-Fourier-level-2’ invariant of the degree-2 SOS moment-matrix proxy) **Field B** (complexity object): Spectral / degree-2 SOS moment-matrix relaxation of Max-Cut on random 3-regular graphs (Delorme-Poljak spectral upper bound $(n/4)\lambda_{\max}$ as the canonical degree-2 SOS surrogate; integrality gap target ≥ 0.121)

Statement:

For a 3-regular graph G on n vertices with Laplacian $L = D - A$ having eigenvalues $0 = \lambda_n < \lambda_{n-1} \leq \dots \leq \lambda_1$, define the hypercontractive Laplacian flatness $\text{HCF}(G) := (n-1) \cdot (\sum_{k=1}^{n-1} \lambda_k^4) / (\sum_{k=1}^{n-1} \lambda_k^2)^2 \in [1, n-1]$, and the spectral degree-2 moment-matrix Max-Cut ratio $\rho(G) := \text{MC}(G) / \text{SB}(G)$, where $\text{SB}(G) := (n/4) \cdot \lambda_1$ and $\text{MC}(G)$ is the true Max-Cut. Conjecture: for every 3-regular graph G with $n \geq 12$, $\rho(G) \leq 1 - (1/10) \cdot ((\text{HCF}(G) - 1)/(n-1))^{1/2}$. A single 3-regular graph violating this strict inequality refutes the conjecture; in particular, any spectral-SOS 0.879-approximator would require $\text{HCF}(G) < 1 + (10 \cdot 0.121)^2 \cdot (n-1)$.

Rationale (proposer’s reasoning):

The Laplacian spectrum is exactly the eigenvalue spectrum of the rank-1 degree-2 SOS moment-matrix surrogate $B \propto v_1 v_1^T$ scaled by λ_1 , and its 4-vs-2-norm ratio is the natural Boolean hypercontractive flatness invariant; a spectrum concentrated at the top ($\text{HCF} \approx 1$) corresponds to a ‘junta-like’ moment matrix where the spectral relaxation is tight, while a spread spectrum ($\text{HCF} \gg 1$) forces an integrality gap exactly because the dominant eigenvector loses bipartite alignment with G ’s edges. Restricting to 3-regular graphs avoids the dense-graph (K_n) and exact-cycle pathologies where the spectral bound is tight identically. The bridge between hypercontractive flatness of Laplacian spectra and Max-Cut integrality gaps appears to have < 5 papers (Friedgut/Bourgain-style hypercontractivity has not been formally aimed at SOS Max-Cut moment matrices).

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d752374825016f37

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 150 random 3-regular graphs (30 each for n in $\{12, 14, 16, 18, 20\}$), the conjecture is SUPPORTED iff every instance satisfies $\rho(G) \leq U(G)$ (support_fraction = 1.0) AND the minimum margin $\min(U(G) - \rho(G)) \geq 0$; any single instance with $\rho(G) > U(G)$ FALSIFIES it.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL PROOFS	SAFE	0.97	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Delorme Poljak spectral Max-Cut bound 3-regular integrality gap - hypercontractivity Laplacian eigenvalue fourth moment Bonami Beckner cubic graph - degree-2 sum-of-squares Max-Cut spectral relaxation cubic graph integrality gap 0.878

Top relevant hits considered: - [<http://arxiv.org/abs/1107.1630v3>] On the Integrality Gap of the Subtour LP for the 1,2-TSP - [<http://arxiv.org/abs/1207.5696v1>] Beyond Max-Cut: λ -Extendible Properties Parameterized Above the Poljak-Turzik Bound - [<http://arxiv.org/abs/2403.19550v2>] Spectral gap for surfaces of infinite volume with negative curvature - [s2:1911.01411] Lifting sum-of-squares lower bounds: degree-2 to degree-4

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from fractions import Fraction

def matrix_multiply(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)
    result = [[0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
```

```

        for j in range(m):
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_transpose(A):
    return [list(row) for row in zip(*A)]

def matrix_norm(A):
    return math.sqrt(sum(sum(a**2 for a in row) for row in A))

def matrix_eigenvalues(A):
    n = len(A)
    if n == 1:
        return [A[0][0]]

    # QR decomposition
    Q = [[Fraction(0) for _ in range(n)] for _ in range(n)]
    R = [[Fraction(0) for _ in range(n)] for _ in range(n)]

    for j in range(n):
        v = [Fraction(0) for _ in range(n)]
        for i in range(n):
            v[i] = A[i][j]
            for k in range(j):
                v[i] -= R[k][j] * Q[i][k]

        norm = matrix_norm([v])
        if norm == 0:
            continue

        for i in range(n):
            Q[i][j] = Fraction(v[i], norm)

        for k in range(j, n):
            R[j][k] = sum(Q[i][j] * A[i][k] for i in range(n))

    # Compute eigenvalues of R
    eigenvalues_R = []
    for i in range(n):
        eigenvalues_R.append(R[i][i])

    return eigenvalues_R

def generate_3_regular_graph(n, seed):
    random.seed(seed)
    if n % 2 != 0:
        raise ValueError("n must be even for a 3-regular graph")

    edges = []
    stubs = list(range(n)) * 3

    while stubs:
        u = random.choice(stubs)
        stubs.remove(u)
        v = random.choice([s for s in stubs if s != u])

```

```

        stubs.remove(v)
        edges.append((u, v))

# Check for multi-edges and self-loops
    edge_set = set()
    for u, v in edges:
        if u == v or (v, u) in edge_set:
            return None
        edge_set.add((u, v))

    return edges

def compute_laplacian(edges, n):
    A = [[0 for _ in range(n)] for _ in range(n)]
    D = [[0 for _ in range(n)] for _ in range(n)]

    for u, v in edges:
        A[u][v] = 1
        A[v][u] = 1
        D[u][u] += 1
        D[v][v] += 1

    L = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            L[i][j] = D[i][j] - A[i][j]

    return L

def compute_hcf(L, eigenvalues):
    n = len(L)
    sum_lambda_4 = sum(eigenvalue**4 for eigenvalue in eigenvalues[1:])
    sum_lambda_2 = sum(eigenvalue**2 for eigenvalue in eigenvalues[1:])
    if sum_lambda_2 == 0:
        return 1.0
    hcf = (n - 1) * sum_lambda_4 / (sum_lambda_2**2)
    return max(1.0, min(hcf, n - 1))

def compute_sb(L, eigenvalues):
    n = len(L)
    lambda_1 = eigenvalues[-1]
    return (n / 4) * lambda_1

def compute_mc(edges, n):
    max_cut = 0
    for subset in itertools.product([0, 1], repeat=n):
        cut_size = 0
        for u, v in edges:
            if subset[u] != subset[v]:
                cut_size += 1
        if cut_size > max_cut:
            max_cut = cut_size
    return max_cut

def run_trial(seed):
    n_values = [12, 14, 16, 18, 20]

```

```

metric_values = []
instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5): # Generate 5 graphs per n
        graph = None
        while graph is None:
            graph = generate_3_regular_graph(n, seed)
            seed += 1

        L = compute_laplacian(graph, n)
        eigenvalues = sorted(matrix_eigenvalues(L))

        hcf = compute_hcf(L, eigenvalues)
        sb = compute_sb(L, eigenvalues)
        mc = compute_mc(graph, n)

        if sb == 0:
            continue

        rho = mc / sb
        upper_bound = 1 - (1/10) * math.sqrt((hcf - 1) / (n - 1))

        if rho > upper_bound:
            conjecture_holds = False
            counterexample = f"n={n}, rho={rho}, upper_bound={upper_bound}, hcf={hcf}, sb={sb}"

        metric_values.append(upper_bound - rho)
        instances_tested += 1

if not metric_values:
    return {
        "metric_name": "margin",
        "metric_value": 0.0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No valid instances generated"
    }

mean_metric = sum(metric_values) / len(metric_values)
return {
    "metric_name": "margin",
    "metric_value": mean_metric,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,

    metric_values = []
    instances_tested = 0
    conjecture_holds_all = True

```

```

counterexample = ""

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    metric_values.append(result["metric_value"])
    instances_tested += result["instances_tested"]
    if not result["conjecture_holds"]:
        conjecture_holds_all = False
        counterexample = result["counterexample"]

if not metric_values:
    print("RESULT: INCONCLUSIVE reason=no_valid_instances")
    sys.exit(0)

mean_metric = sum(metric_values) / len(metric_values)
std_metric = math.sqrt(sum((x - mean_metric)**2 for x in metric_values) / len(metric_values))
support_fraction = sum(1 for x in metric_values if x >= 0) / len(metric_values)

if conjecture_holds_all and support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
elif not conjecture_holds_all:
    first_failing_seed = seeds[next(i for i, x in enumerate(metric_values) if x < 0)]
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67b46128.py", line 194, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67b46128.py", line 148, in run_trial
    eigenvalues = sorted(matrix_eigenvalues(L))
                    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_67b46128.py", line 55, in matrix_eigenvalues
    Q[i][j] = Fraction(v[i], norm)
                ~~~~~
  File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be ")
TypeError: both arguments should be Rational instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` in `matrix_eigenvalues` before any of the 150 instances were evaluated, so no data exists to assess `support_fraction` or `min_margin`. Per rule 2, a crash yields INCONCLUSIVE. | next: Fix `matrix_eigenvalues` to use floating-point (`numpy.linalg.eigvalsh`) or coerce the norm to a Rational/sqrt-friendly type before re-running the 150-instance sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	224104
2	preregistration	claude_max	opus	0	0	5121
3	novelty	claude_max	opus	0	0	4625
4	novelty	claude_max	opus	0	0	10617
5	test_gen	mistral	codestral-latest	0	0	320046
6	test_gen	mistral	codestral-latest	0	0	308990
7	test_gen	mistral	codestral-latest	0	0	311144
8	test_gen	mistral	codestral-latest	0	0	315756
9	judge	claude_max	opus	0	0	4464

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1504868 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 4}

(full prompt+response transcripts available in `research/audit/c8a7a640d375.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c8a7a640d375.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/c8a7a640d375.tar.gz` (if generated)